CodeMentor — Infrastructure Requirements & Setup Guide (Srinivas's copy)

Last updated: 2026-10-02 **Audience:** you, as the sole maintainer, deciding what to build and in what order. **Companion document:** a version of this aimed at HOD/VC exists as a shareable doc (link given in chat) — that one argues the case; this one tells you how to actually do it. Don't duplicate edits between them by hand; if a number changes, update both.

This consolidates the scale-readiness programme (`docs/scale-readiness/`, 21 chapters, written 2026-08-09) and `docs/DEPLOYMENT_GUIDE.md` (2026-09-22) against the scale targets raised in the 2026-10-01 review (`Review_Feedback.md` items 1–4, 7). It doesn't re-derive anything those documents already settled — it points at them and gives you the practical "do this" version.

---

1. Where you actually stand today (don't lose track of this)

- **Live demo:** Vercel (frontend) + Render free tier (backend) + Judge0's public demo API (`ce.judge0.com`) for code execution. Cost: **$0/month**. This is "Path C" in the deployment guide — fine for a demo, not sized for real classroom traffic.
- **No college server access yet.** Everything below about college-server hosting is the *plan*, not the current state.
- **No GCP VM currently running** — it was a temporary demo instance and was torn down. Judge0 execution on your *own* infrastructure is not live anywhere right now.
- This guide is about what changes **when real classroom-scale traffic arrives** — i.e., when Path C's free-tier limits (Render's cold starts, the public Judge0 API's rate limits) stop being acceptable.

---

2. The software stack (what's already built — listed so you know what you're running)

Layer	Technology	Where it runs	Needs from you
Frontend	Next.js 16 + MUI v6 (Material 3), Monaco editor	Vercel (free tier)	Nothing — stays here regardless of path
Backend	Node.js 22 + Express 5, stateless	Render (free) or Railway	Node 22, ~512MB RAM, **Java 17** (for JPlag)
Database	Firebase Firestore	Google, managed	Nothing — free tier covers this scale
Auth	Firebase Auth → backend mints its own JWT	Google, managed	Nothing
Cache / rate limiting	Upstash Redis (REST-based)	Upstash, managed	Nothing — free tier: 10k commands/day
Code execution	Judge0 CE, in privileged Docker (`isolate` sandbox)	**Must be a real machine you control**	Linux, Docker, `sudo`/root, 2+ vCPU, 4GB+ RAM minimum
Plagiarism	JPlag 5.1.0 (Java), shelled out from backend	Inside the backend container	Java 17 runtime
AI tutor	Google Gemini via `@google/genai`	Google, managed	API key; free tier then pay-per-token

The one thing that decides everything else: Judge0 executes untrusted student code. That requires Linux cgroups + namespaces + privileged containers — no serverless or managed platform anywhere provides this. Every hosting decision below is really just "where does Judge0 run," because everything else already runs free.

---

3. Hardware sizing — two different numbers for two different purposes

Don't confuse these — they answer different questions.

3a. Minimum to stand up Path B at all (get it running, low/no real load)

4 vCPU, 8 GB RAM, 100 GB disk, Ubuntu 22.04 LTS, Docker + sudo. This runs the full stack (frontend container not needed — stays on Vercel; backend + Judge0 + Judge0's own Postgres/Redis) for development, testing, or a handful of concurrent users. This is the number in `docs/DEPLOYMENT_GUIDE.md` Part 4 §B1.

3b. Sized for the real target: 1,000 concurrent students, 3-hour exam

16 CPU cores (20–24 preferred), 32 GB RAM, 100 GB disk. This is derived, not guessed — full working in `docs/scale-readiness/09-judge0-capacity.md`:

```
1,000 students × ~60 executions each (3h exam) = 60,000 executions
÷ 10,800 seconds session                       = 5.6 executions/sec
× 1.33 CPU-seconds per execution (mixed C/C++/Java/Python) = 7.4 vCPU sustained
× 2 (peak clustering near deadlines)           = ~15 vCPU
× 1.3 (safety margin for estimation error)      = ~20 vCPU → ask for 16, be glad
with 24
```

Your 200-simultaneous-submissions target (Review_Feedback item 1) is a fifth of this and needs no separate sizing — it's already inside the 1,000-student model.

RAM: 16 execution sandboxes × 512 MB + Judge0/Postgres/Redis/OS overhead → 32 GB. **Disk:** Judge0's own Postgres grows with submission history → 100 GB, revisit annually.

What swings this number 6×: the language mix. Java costs ~9× Python per execution (compilation dominates). If CRT/placement sessions skew Java-heavy, size toward 38 cores; if they skew Python, 6 cores suffices. Know your actual exam's language mix before an exam, not after.

3c. GPU

None, ever, for Judge0. It's CPU-only. If the college eventually gives you access to the H100 server mentioned in your notes, it's wasted on code execution — its only genuine use case for this project is self-hosting the AI tutor (Ollama or vLLM) to drop the Gemini API dependency. Do that *after* the basic deployment is stable, not during.

---

4. Network requirements (the part IT/sysadmin will actually push back on)

- **Outbound HTTPS (443) only.** That's all you need, via Cloudflare Tunnel.
- **No public IP. No inbound firewall rules. No port forwarding.** Say this explicitly to whoever controls the server — it's the single most reassuring fact you have, and it removes most of their objection before they raise it.
- Judge0 itself is never exposed publicly — it sits on an isolated Docker network, reachable only by your backend.

Setting up the tunnel (once you have a machine)

```
curl -L
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
linux-amd64.deb -o cloudflared.deb
sudo dpkg -i cloudflared.deb
cloudflared tunnel login
cloudflared tunnel create codementor
```

`~/.cloudflared/config.yml`:

```
tunnel: <tunnel-UUID>
credentials-file: /home/youruser/.cloudflared/<tunnel-UUID>.json
ingress:
  - hostname: judge0.yourdomain.com
    service: http://localhost:2358
  - hostname: api.yourdomain.com
    service: http://localhost:3001
  - service: http_status:404
```

```
cloudflared tunnel route dns codementor judge0.yourdomain.com
cloudflared tunnel route dns codementor api.yourdomain.com
sudo cloudflared service install
```

You need a domain on Cloudflare for this (~₹900/yr) — the one unavoidable cost of the college-server path.

---

5. Deploying the stack once you have a machine

```
git clone https://github.com/admin-codementor/codementor.git
cd codementor
cp backend/.env.example backend/.env
nano backend/.env    # fill in every value
docker compose up -d --build
```

This brings up backend, Judge0 server, Judge0 worker, and Judge0's own Postgres + Redis (frontend stays on Vercel). Then point `cloudflared` at ports 3000/3001 as configured above.

Judge0 config — don't skip this, the hardware is wasted without it

`judge0/judge0.conf`:

Setting	Current repo default	Set to	Why
`COUNT_OF_RUN_WORKERS`	2	**= your core count (16)**	🔴 This is the actual concurrency limit — a 32-core machine with `workers=2` performs identically to a 2-core machine. Highest-value single line in the whole system.

Setting	Current repo default	Set to	Why
`MAX_QUEUE_SIZE`	100	**2000**	At 100, the 101st submission gets a 503 with no retry. Raising it converts overload into a wait, not a failure.
`MAX_MEMORY_LIMIT`	1 GB	**512 MB**	16 workers × 1GB = 16GB worst case; 512MB is ample per student program and halves memory pressure
`ENABLE_NETWORK`	false	**keep false**	Student code gets zero network access
`AUTHN_TOKEN`	empty in repo	**long random value**	Verify the deployed instance actually has this set

Keep Firebase/Firestore, Auth, and Upstash as-is even after self-hosting Judge0 — they're free at your scale and rewriting to self-hosted Postgres is weeks of work for no benefit (see `docs/scale-readiness/13-hosting-decision.md`).

---

6. Migration sequence (every step independently reversible)

Step	Move	Rollback
1	Judge0 → college server	Change `JUDGE0_URL` back. One env var.
2	Run both (public API + self-hosted) in parallel 3–5 days	—
3	Decommission reliance on the public API	—
4	Backend → college server (if desired)	Change `API_PROXY_TARGET`, redeploy

Judge0 moves first because its rollback is the cheapest in the entire stack — a single environment variable. Don't move the backend until Judge0 has been stable for a few days.

---

7. Security — what to actually do, right-sized for this scale

Full detail: `docs/scale-readiness/05-auth-and-security.md`. The honest summary: this system was built with real security awareness already (Firebase handles passwords, untrusted code is sandboxed, there's audit logging, input validation on the submit path). What's open, in priority order:

#	Finding	Action	Time
1	🔴 Firestore rules unverified	Check today — set deny-all, access only through backend	10 min
2	🔴 Firebase still on Spark (free) plan	Upgrade to Blaze + set a budget alert — Spark's 50k reads/day cap will take the whole app down ~20 min into a real session	10 min
3	🟠 Fail-open submit rate limiter	Make it fail closed on the submission endpoint specifically	PR-sized
4	🟠 `trust proxy` config may break exam IP restriction	Verify against real logs	15 min
5	🟠 7-day token refresh, no revocation	Shorten lifetime, add a `tokenVersion` field	PR-sized
6	🟡 No CSP in production	Enable it	PR-sized

On Kubernetes / API gateway / pod security standards / full RBAC infra (raised in the Oct review): disproportionate at this scale — see the reasoning in the HOD/VC doc. Docker Compose with explicit per-container CPU/memory limits gives you the same isolation guarantee a Kubernetes NetworkPolicy would, without taking on cluster operations as a second job. Revisit only if the college's own security policy mandates it regardless of scale, or if you genuinely outgrow one machine.

---

8. Cost model (so you can sanity-check any future quote against reality)

Option	Annual cost	Notes
CodeTantra (current vendor)	~₹15,00,000	~3,000 users, baseline
Path A — commercial cloud (GCP + Railway)	~₹4,20,000 (or ~₹2,00,000 with exam-day resizing)	No IT dependency, costs real money forever
Path B — college server	~₹15,000	Domain name is nearly the whole cost
Path C — current demo	₹0	Not sized for real traffic; what's live right now

Full derivation and the honest caveats (support, continuity risk if you graduate, labour cost) in `docs/scale-readiness/08-cost-model.md`.

---

9. What to actually do, in order

1. **Verify items in §7** — the two 🔴 rows especially, they're ~20 minutes total and remove catastrophic failure modes that exist regardless of hosting path.
2. **Get the college server conversation started** (the HOD/VC doc is the artifact for this) — specs, OS, who administers it, uptime/reimaging policy. Four questions in `DEPLOYMENT_GUIDE.md` §B1 decide whether this path is even viable; get answers before investing a weekend in migration.
3. **While waiting on server access:** if the public Judge0 API's rate limits start causing real problems, a paid Judge0 API key is the fastest unblock — same `JUDGE0_URL` swap, no infrastructure change.
4. **Once you have a machine:** follow §5 exactly, especially the Judge0 config table — the hardware is wasted without `COUNT_OF_RUN_WORKERS` matching core count.
5. **Load test before any real exam** against the assumptions in §3b — none of those numbers (executions/student, CPU per language) have been measured on the real system yet. `docs/scale-readiness/19-load-testing.md` has the plan.
6. **Phase the rollout** — one class, then one year, then the department. Don't let the first real run be at full peak load.

Where to go for more depth

Every number above is derived in full in `docs/scale-readiness/`:

- Load model and bottlenecks: chapters 6–7
- Cost model: chapter 8
- Judge0 capacity derivation: chapter 9
- Hosting decision record: chapter 13
- Deployment runbook, monitoring, incident playbooks: chapters 16–18
- Load testing plan: chapter 19
- The original server-request package (sysadmin-facing): chapter 20

Don't re-derive what's already there — update the relevant chapter if a real measurement changes an assumption, and update this guide's summary to match.